Data Structures (CSN12101)
ASSIGNMENT-1

Name : Aayansh Niranjan Date : 13/01/2026
Reg_No.: 20253004

Q1. Write a function sumdiv( ) that finds the sum of divisors of a number. (Divisors of a number are those numbers by which the number is divisible). For example, divisors of 36 are 1, 2, 3, 4, 6, 9,18. A number is called perfect if the sum of divisors of that number is equal to the number. For example, 28 is a perfect number, since 1+2+4+7+14 =28. Write a program to print all the perfect numbers in a given range.

A1. CODE:

```
#include<stdio.h>

int sumdiv(int n){
  int count=0;
  for(int i=1;i*i<=n;i++){
    if(n%i==0){
      if(i!=n) count+=i;
      if(i*i!=n && i!=1) count+=n/i;
    }
  }
  return count;
}
int main(){
  int n;
  printf("Enter the number : ");
  scanf("%d",&n);
  int count=sumdiv(n);
  //printf("%d",count);
  if(count==n){printf("The number is a perfect number\n");}
  else{printf("The number is not a perfect number\n");}
  return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004$ ./p1
Enter the number : 28
The number is a perfect number
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004$ ./p1
Enter the number : 36
The number is not a perfect number
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004$
```

Q2. You are supposed to entered positive integer through the keyboard; you have to find the binary equivalent of this number using recursion.
A2. CODE:

```
#include <stdio.h>

void binary(int n){
    if(n==0)
      return;
    binary(n/2);
    printf("%d",n%2);
}
int main(){
    int num;
    printf("Enter a positive integer: ");
    scanf("%d",&num);
    if (num==0) printf("Binary equivalent: 0");
    else{
        printf("Binary equivalent: ");
        binary(num);
    }
    printf("\n");
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004$ ./p2
Enter a positive integer: 12
Binary equivalent: 1100
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004$ ./p2
Enter a positive integer: 342345
Binary equivalent: 1010011100101001001
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004$
```

Q3. Write a program to find the value of one number raised to the power of another number (do not use pow() function).
A3. CODE:

```
#include <stdio.h>

void binary(int n,int b){
    if(n==0)
      return;
    binary(n/b,b);
    printf("%d",n%b);
}
int main(){
    int num,base;
    printf("Enter a positive integer: ");
    scanf("%d",&num);
    printf("Enter the base: ");
    scanf("%d",&base);
    if (num==0) printf("The number 0 in base %d is : 0",base);
    else printf("The number %d in base %d is : ",num,base);
    binary(num,base);
    printf("\n");
    return 0;
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_1$ ./p3
Enter a positive integer: 64
Enter the base: 2
The number 64 in base 2 is : 1000000
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_1$ ./p3
Enter a positive integer: 72
Enter the base: 3
The number 72 in base 3 is : 2200
```

Q4.Write a program in C to perform the Addition and Subtraction of two matrices.
A4. CODE :

```
#include<stdio.h>
int main(){
  int row;
  int col;
  printf("Enter the dimensions of the matrices : ");
  scanf("%d %d",&row,&col);
  int A[row][col];
  int B[row][col];
  printf("Enter the first matrix : \n");
  for(int i=0;i<row;i++){
    for(int j=0;j<col;j++){
      scanf("%d",&A[i][j]);
    }
  }
  printf("Enter the second matrix : \n");
  for(int i=0;i<row;i++){
    for(int j=0;j<col;j++){
      scanf("%d",&B[i][j]);
    }
  }
  printf("Addition matrix : \n");
  for(int i=0;i<row;i++){
    for(int j=0;j<col;j++){
      printf("%d ",A[i][j]+B[i][j]);
    }
    printf("\n");
  }
  printf("Subtraction matrix : \n");
  for(int i=0;i<row;i++){
    for(int j=0;j<col;j++){
      printf("%d ",A[i][j]-B[i][j]);
    }
    printf("\n");
  }
  return 0;
}
```

OUTPUT :

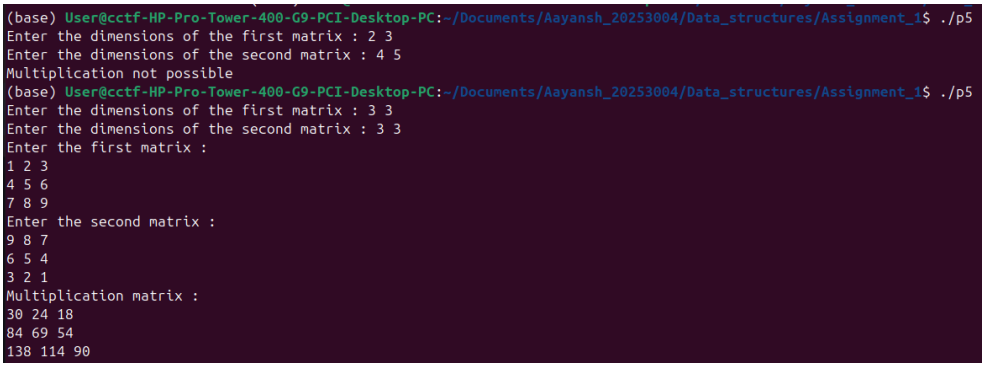
```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_1$ ./p4
Enter the dimensions of the matrices : 2 3
Enter the first matrix :
1 2 3
4 5 6
Enter the second matrix :
7 8 9
10 11 12
Addition matrix :
8 10 12
14 16 18
Subtraction matrix :
-6 -6 -6
-6 -6 -6
```

Q5. Write a program in C to perform Matrix Multiplication.
A5. CODE :

```
#include<stdio.h>

int main(){
  int row1,row2;
  int col1,col2;
  printf("Enter the dimensions of the first matrix : ");
  scanf("%d %d",&row1,&col1);
  printf("Enter the dimensions of the second matrix : ");
  scanf("%d %d",&row2,&col2);
  if(col1!=row2){printf("Multiplication not possible\n");return 0;}
  int A[row1][col1];
  int B[row2][col2];
  printf("Enter the first matrix : \n");
  for(int i=0;i<row1;i++){
    for(int j=0;j<col1;j++){
      scanf("%d",&A[i][j]);
    }
  }
  printf("Enter the second matrix : \n");
  for(int i=0;i<row2;i++){
    for(int j=0;j<col2;j++){
      scanf("%d",&B[i][j]);
    }
  }
  printf("Multiplication matrix : \n");
  for(int i=0;i<row1;i++){
    for(int j=0;j<col2;j++){
      int count=0;
      for(int k=0;k<col1;k++){
        count+=A[i][k]*B[k][j];
      }
      printf("%d ",count);
    }
    printf("\n");
  }
  return 0;
}
```

OUTPUT :



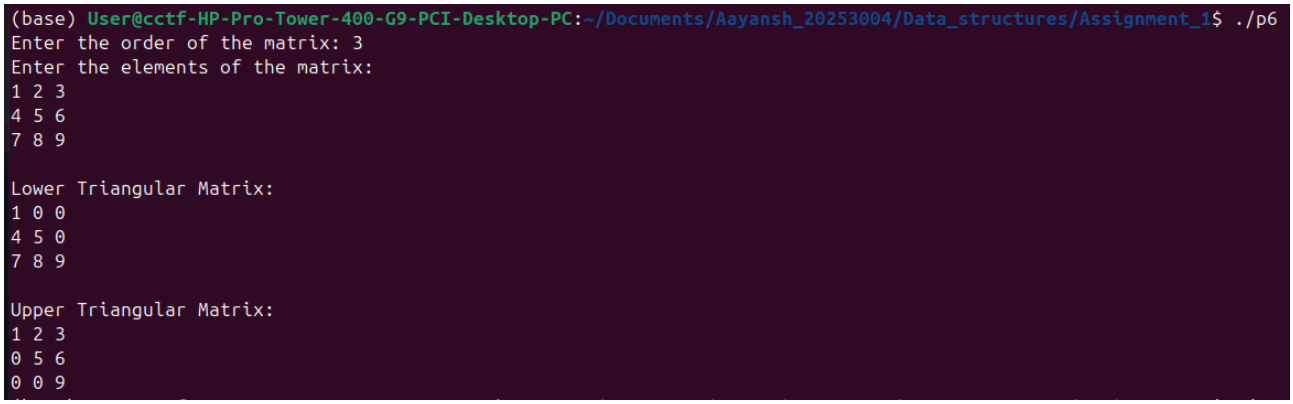
Q6.Write a program in C to display the lower triangular matrix of an array and upper triangular matrix of an array.

A6. CODE:

```
#include <stdio.h>

int main() {
    int i,j,n;
    printf("Enter the order of the matrix: ");
    scanf("%d",&n);
    int a[n][n];
    printf("Enter the elements of the matrix:\n");
    for(i=0;i<n;i++){
      for(j=0;j<n;j++) scanf("%d",&a[i][j]);
    }
    printf("\nLower Triangular Matrix:\n");
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            if(i>=j) printf("%d ",a[i][j]);
            else printf("0 ");
        }
        printf("\n");
    }
    printf("\nUpper Triangular Matrix:\n");
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            if(i<=j) printf("%d ", a[i][j]);
            else printf("0 ");
        }
        printf("\n");
    }
    return 0;
}
```

OUTPUT:



Q7.Write a C program to read and print student details using structure pointer, demonstrate example of structure with pointer.
A7. CODE:

```
#include<stdio.h>

struct student{
  int roll;
  int marks;
};

int main(){
  struct student* Aayansh;
  printf("Enter the roll number");
  scanf("%d",&(Aayansh->roll));
  printf("Enter the marks");
  scanf("%d",&(Aayansh->marks));
  printf("Roll number : %d\n",Aayansh->roll);
  printf("Marks : %d\n",Aayansh->marks);
  }
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_1$ ./p7
Enter the roll number1
Enter the marks99
Roll number : 1
```

Q8. Implement a binary file database of records with structures:
• Structure Employee { int id; char name[40]; double salary; };
• Provide functions to append, search by id, update salary, and delete logically (mark as inactive), all using fread/fwrite on a binary file.
A8. CODE:

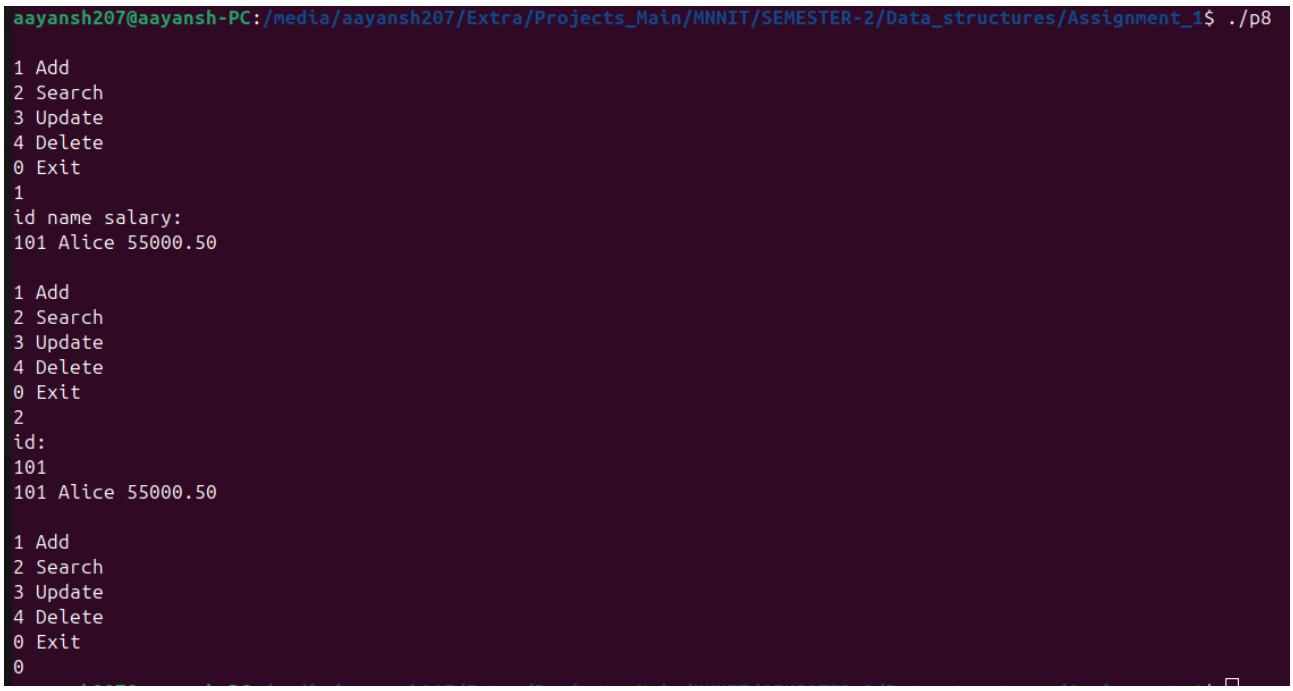
```
#include<stdio.h>

struct Employee{
    int id;
    char name[40];
    double salary;
    int active;
};
int main(){
    FILE *fp;
    struct Employee e;
    int ch,id;
    double sal;

    while(1){
        printf("\n1 Add\n2 Search\n3 Update\n4 Delete\n0 Exit\n");
        scanf("%d",&ch);
        if(ch==0)break;
```

```
    if(ch==1){
       fp=fopen("emp.dat","ab");
       printf("id name salary:\n");
       scanf("%d %s %lf",&e.id,e.name,&e.salary);
       e.active=1;
       fwrite(&e,sizeof(e),1,fp);
       fclose(fp);
    }
    else if(ch==2){
       fp=fopen("emp.dat","rb");
       printf("id:\n");
       scanf("%d",&id);
       while(fread(&e,sizeof(e),1,fp)){
          if(e.id==id && e.active)
             printf("%d %s %.2lf\n",e.id,e.name,e.salary);
       }
       fclose(fp);
    }
    else if(ch==3){
       fp=fopen("emp.dat","rb+");
       printf("id newSalary:\n");
       scanf("%d %lf",&id,&sal);
       while(fread(&e,sizeof(e),1,fp)){
          if(e.id==id && e.active){
             e.salary=sal;
             fseek(fp,-sizeof(e),SEEK_CUR);
             fwrite(&e,sizeof(e),1,fp);
             break;
          }
       }
       fclose(fp);
    }
    else if(ch==4){
       fp=fopen("emp.dat","rb+");
       printf("id:\n");
       scanf("%d",&id);
       while(fread(&e,sizeof(e),1,fp)){
          if(e.id==id && e.active){
             e.active=0;
             fseek(fp,-sizeof(e),SEEK_CUR);
             fwrite(&e,sizeof(e),1,fp);
             break;
          }
       }
       fclose(fp);
    }
  }
  return 0;
}
```

OUTPUT:


aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_1$ ./p8
1 Add
2 Search
3 Update
4 Delete
0 Exit
1
id name salary:
101 Alice 55000.50
1 Add
2 Search
3 Update
4 Delete
0 Exit
2
id:
101
101 Alice 55000.50
1 Add
2 Search
3 Update
4 Delete
0 Exit
0

Q9. Write a memory pool allocator using structures and unions:

- Use an array of union Block { struct { union Block *next; } freeNode; char payload[BLOCK_SIZE]; };
- Manage a free list; implement pool_malloc/pool_free that hand out fixed size blocks from the pool.

A9. CODE:

```
#include<stdio.h>
#define BLOCK_SIZE 32
#define POOL_SIZE 10
union Block{
   struct{union Block*next;}freeNode;
   char payload[BLOCK_SIZE];
};
union Block pool[POOL_SIZE];
union Block*freeList;
void pool_init(){
   int i;
   for(i=0;i<POOL_SIZE-1;i++)pool[i].freeNode.next=&pool[i+1];
   pool[POOL_SIZE-1].freeNode.next=NULL;
   freeList=&pool[0];
}
void*pool_malloc(){
   union Block*b;
   if(freeList==NULL)return NULL;
   b=freeList;
   freeList=freeList->freeNode.next;
   return b;
}
void pool_free(void*ptr){
   union Block*b;
   if(ptr==NULL)return;
   b=(union Block*)ptr;
   b->freeNode.next=freeList;
   freeList=b;
}
int main(){
   union Block*p1,*p2;
   pool_init();
   p1=(union Block*)pool_malloc();
   p2=(union Block*)pool_malloc();
   if(p1)sprintf(p1->payload,"Hello");
   if(p2)sprintf(p2->payload,"World");
   printf("%s %s\n",p1->payload,p2->payload);
   pool_free(p1);
   pool_free(p2);
   return 0;
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_1$ ./p9
Hello World
```

Q10.You are given an unsorted array with both positive and negative elements. You have to find the smallest positive number missing from the array.

A10. CODE :

```
#include<stdio.h>
#include<stdlib.h>

int main(){
    int a[]={2,3,7,6,8,-1,-10,15};
    int n=sizeof(a)/sizeof(a[0]);
    int *seen=calloc(n+1,sizeof(int));

    for(int i=0;i<n;i++){
        if(a[i]>0 && a[i]<=n) seen[a[i]]=1;
    }
    for(int i=1;i<=n;i++){
        if(!seen[i]){
            printf("%d\n",i);
            return 0;
        }
    }

    printf("%d\n",n+1);
    return 0;
}
```

OUTPUT :

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_1$ ./p10
1
```

Q11. Imagine you're managing an extensive dataset for a scientific research project. The data contains multiple arrays, each representing a different metric (e.g., temperature readings, pressure readings, etc.). Your task is to implement a program that efficiently copies data from one array to another using pointers, while also handling various edge cases like dynamic memory allocation, ensuring no data loss, and handling arrays of variable lengths. You are also required to handle potential errors in memory allocation and provide detailed feedback to the user, including information about the copying process (such as when data is successfully copied or if an issue arises).

A11. CODE :

```
#include<stdio.h>
#include<stdlib.h>

void copyArray(int *src,int *dest,int n){
    for(int i=0;i<n;i++)
        *(dest+i)=*(src+i);
}

int main(){
    int n;
    printf("Enter number of elements: ");
    scanf("%d",&n);
    int *src=malloc(n*sizeof(int));
    int *dest=malloc(n*sizeof(int));
    if(src==NULL || dest==NULL){
        printf("Memory allocation failed\n");
        return 1;
    }
    printf("Enter elements:\n");
    for(int i=0;i<n;i++) scanf("%d",&src[i]);
    copyArray(src,dest,n);

    printf("Data copied successfully:\n");
    for(int i=0;i<n;i++) printf("%d ",dest[i]);
    free(src);
    free(dest);
    return 0;
}
```

OUTPUT :

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_1$ ./p11
Enter number of elements: 4
Enter elements:
10 20 30 40
Data copied successfully:
10 20 30 40 aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_1$
```

Q12. Write a C program to return multiple values from function using pointers.
A12. CODE:

```
#include<stdio.h>

void calculate(int a,int b,int *sum,int *product){
    *sum=a+b;
    *product=a*b;
}

int main(){
    int a,b,sum,product;
    printf("Enter two numbers: ");
    scanf("%d %d",&a,&b);

    calculate(a,b,&sum,&product);

    printf("Sum = %d\n",sum);
    printf("Product = %d\n",product);
    return 0;
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_1$ ./p12
Enter two numbers: 12 7
Sum = 19
Product = 84
```

Q13. Write a program to implement Linear search in a single dimensional array.
A13. CODE:

```
#include<stdio.h>

int main(){
    int n,key;
    printf("Enter array size: ");
    scanf("%d",&n);

    int arr[n];
    printf("Enter elements:\n");
    for(int i=0;i<n;i++) scanf("%d",&arr[i]);

    printf("Enter element to search: ");
    scanf("%d",&key);

    for(int i=0;i<n;i++){
        if(arr[i]==key){
            printf("Element found at index %d\n",i);
            return 0;
        }
    }

    printf("Element not found\n");
    return 0;
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_1$ ./p13
Enter array size: 5
Enter elements:
15 8 92 34 7
Enter element to search: 92
Element found at index 2
```

Q14. Imagine you are working at a school where the exam scores of students are stored in an array. The students' scores are sorted in ascending order, and you need to help a student find his/her score in the list. To do this, you decide to use the Binary Search Algorithm. Binary search is an efficient way to find an item in a sorted list by repeatedly dividing the search interval in half. If the value is less than the item in the middle of the interval, the search continues in the lower half, or if the value is greater, the search continues in the upper half.

A14. CODE:

```
#include<stdio.h>

int main(){
    int n,key;
    printf("Enter number of students: ");
    scanf("%d",&n);

    int scores[n];
    printf("Enter sorted scores:\n");
    for(int i=0;i<n;i++) scanf("%d",&scores[i]);

    printf("Enter score to search: ");
    scanf("%d",&key);
    int low=0,high=n-1;
    while(low<=high){
        int mid=(low+high)/2;

        if(scores[mid]==key){
            printf("Score found at position %d\n",mid);
            return 0;
        }
        else if(key<scores[mid])
            high=mid-1;
        else
            low=mid+1;
    }

    printf("Score not found\n");
    return 0;
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_1$ ./p14
Enter number of students: 6
Enter sorted scores:
10 25 40 55 70 85
Enter score to search: 55
Score found at position 3
```

Q15. Suppose you are working in an organization where the employee's salary is stored in an array. However, the salary are not in order, and you need to sort them from the lowest to the highest. As a part of the task, you decide to implement the Bubble Sort Algorithm. This algorithm repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The process continues until the array is sorted. You also have to find the number of comparisons and time required to sort the array.

A15. CODE:

```
#include<stdio.h>
#include<time.h>

int main(){
    int n;
    printf("Enter number of employees: ");
    scanf("%d",&n);
    int salary[n];
    printf("Enter salaries:\n");
    for(int i=0;i<n;i++) scanf("%d",&salary[i]);
    int comparisons=0;
    clock_t start=clock();
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            comparisons++;
            if(salary[j]>salary[j+1]){
                int temp=salary[j];
                salary[j]=salary[j+1];
                salary[j+1]=temp;
            }
        }
    }
    clock_t end=clock();
    double time_taken=(double)(end-start)/CLOCKS_PER_SEC;
    printf("Sorted salaries:\n");
    for(int i=0;i<n;i++) printf("%d ",salary[i]);
    printf("\nComparisons: %d",comparisons);
    printf("\nTime taken: %f seconds\n",time_taken);
    return 0;
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_1$ ./p15
Enter number of employees: 5
Enter salaries:
5000 1200 3500 800 4200
Sorted salaries:
800 1200 3500 4200 5000
Comparisons: 10
Time taken: 0.000004 seconds
```